
Microsoft
STUDENT AMBASSADOR

Microsoft
Student Ambassadors

Orchestration using Airflow

Session 2

Our Aginda

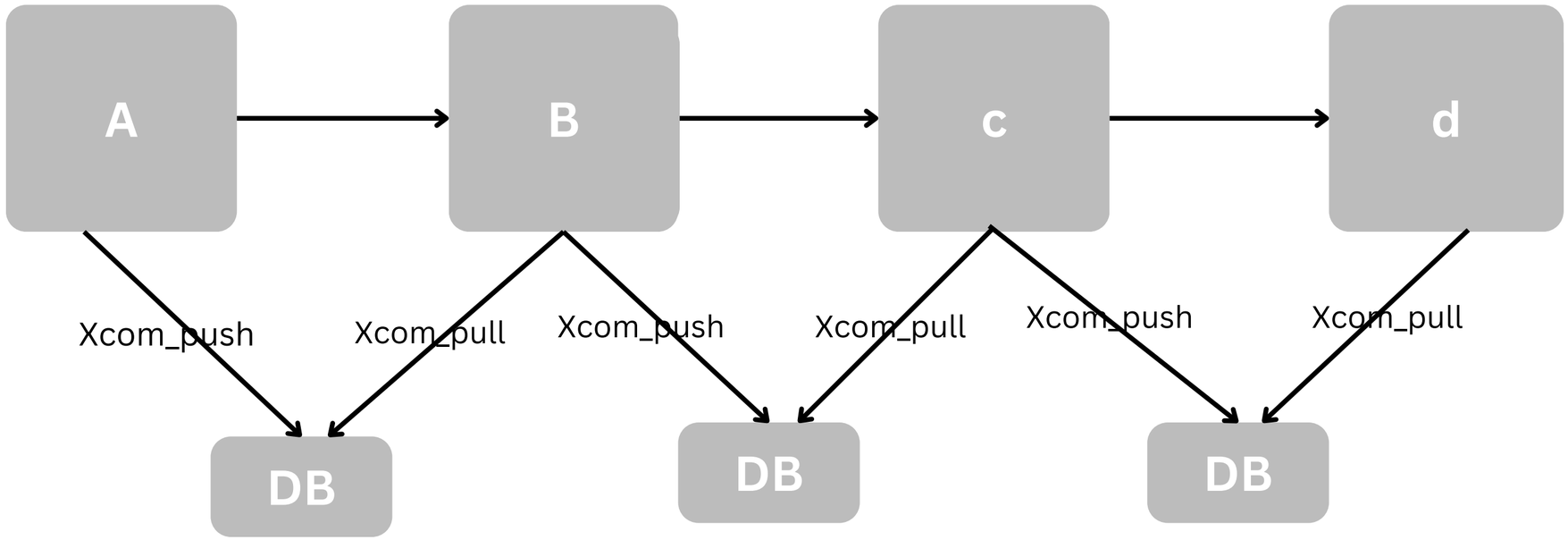
1. Xcom
2. Assets
3. Grouping
4. Branching

---

5. Small ETL

Xcom

- XCom stands for Cross-Communication
- Used to allow Tasks to communicate with each other



How XComs Work

Each XCom is identified by:

- Key (name of the data)
- task_id
- dag_id

XCom Operations

Data is explicitly transferred using:

- xcom_push() → send data
- xcom_pull() → retrieve data

XCom Size Limits by Database Backend

- MySQL: ~64 KB (Very small).
- PostgreSQL: ~1 GB.
- SQLite: ~2 GB

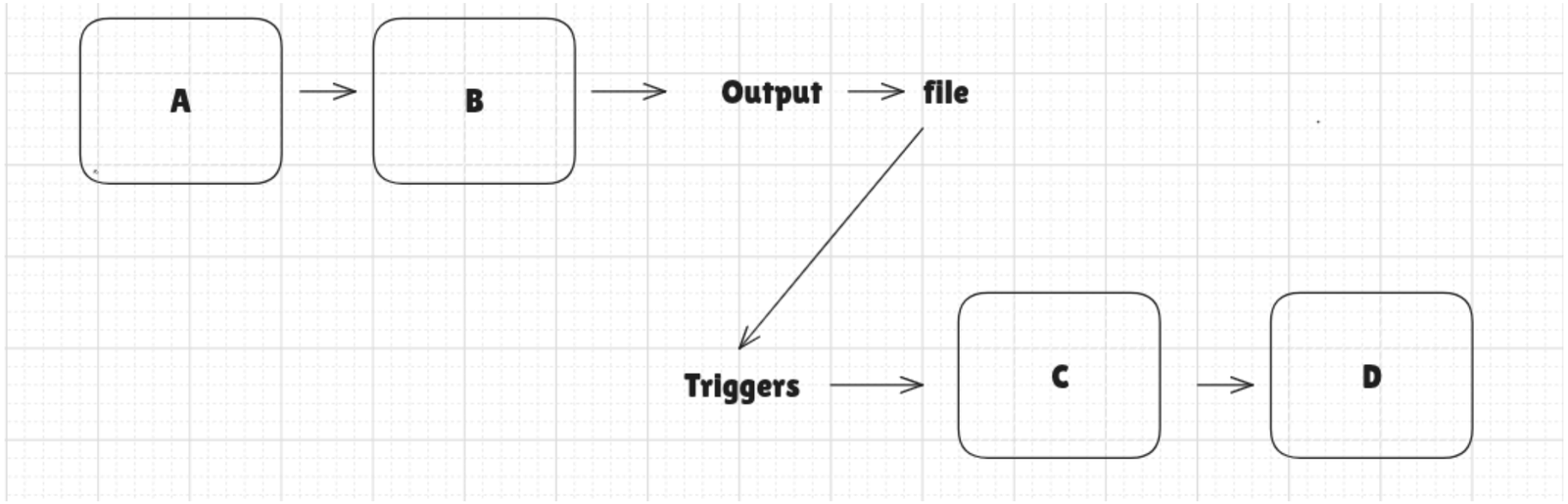
What’s an Asset

An Asset is a collection of related data:

- A table in a DB
- A persisted ML model
- A report
- A directory
- The output of an API
- A file

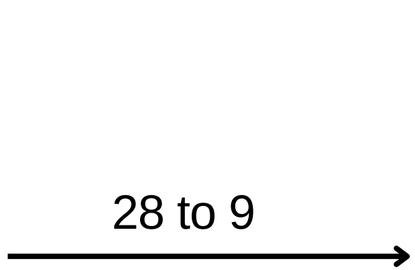
You might find it familiar if you've used Airflow 2.0, where the term "datasets" was commonly used for assets. In Airflow 3.0, however, this has been simplified to just "asset."

Why to use Asset



```python
from airflow.datasets import Dataset
from airflow.decorators import dag, task
from pendulum import datetime

daily_sales = Dataset(uri="daily_sales")
monthly_sales = Dataset(uri="monthly_sales")

@dag(
    schedule="@daily",
    start_date=datetime(2025, 1, 1),
)
def daily_report():

    @task(outlets=daily_sales)
    def extract_sales_data():
        # do some work
        return daily_sales

    extract_sales_data()

@dag(
    schedule=[daily_sales],
    start_date=datetime(2025, 1, 1)
    )
def monthly_report():

    @task(outlets=monthly_sales)
    def aggregate_montly():
        daily_sales = fetch_daily_sales()
        # do some work
        return monthly_sales()

    aggregate_montly()

daily_report()
montly_report()
```

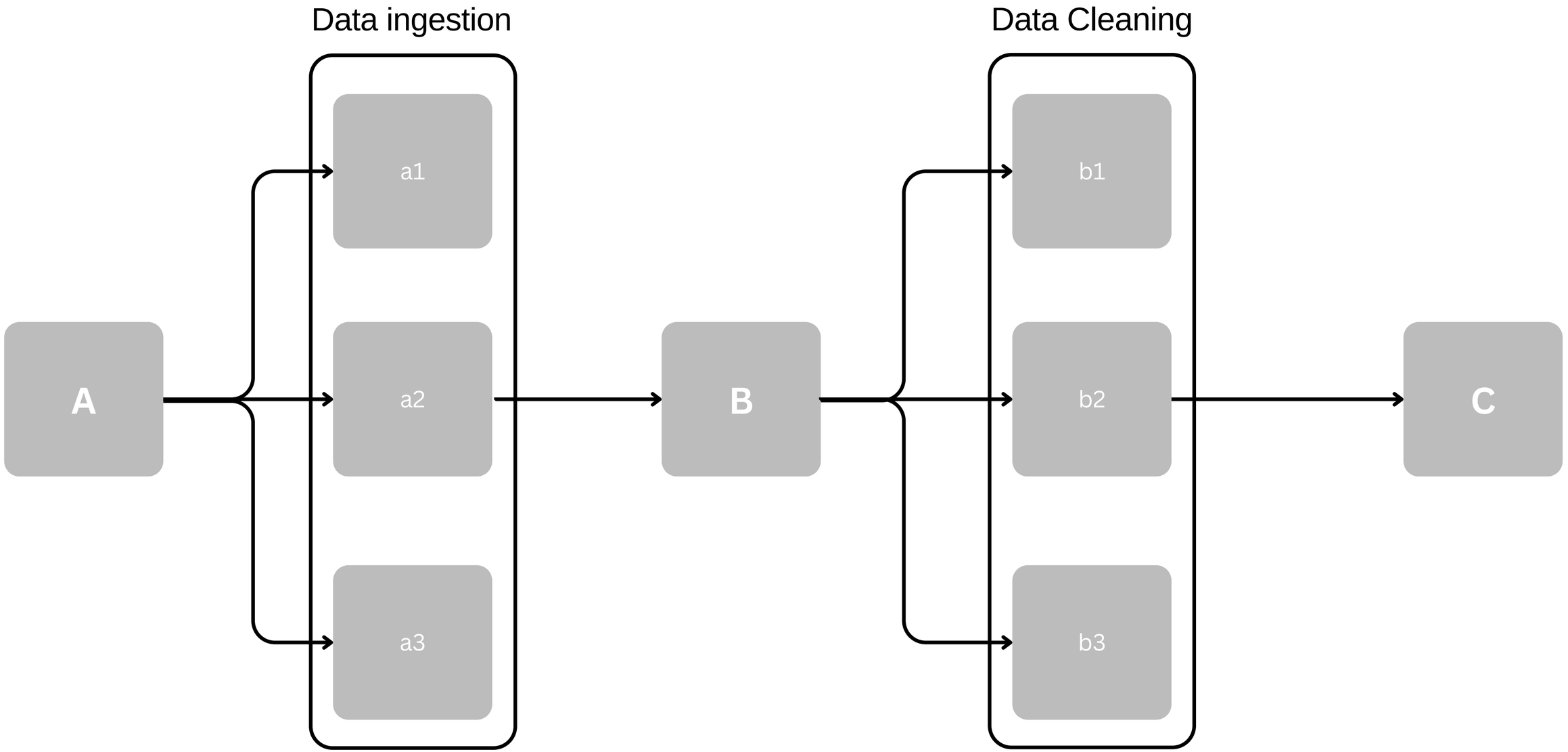
28 to 9



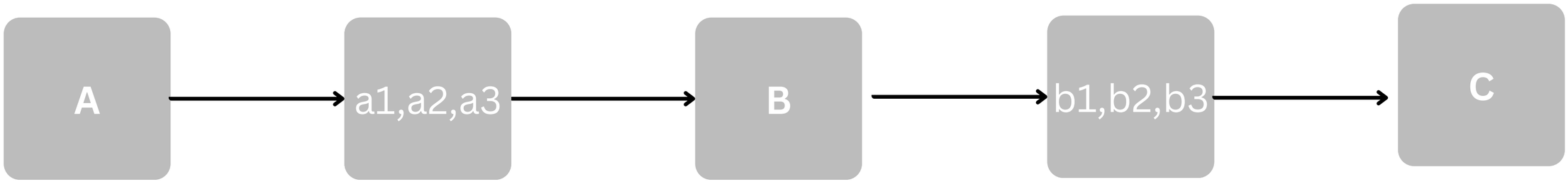
```python
from airflow.sdk.definitions.asset.decorators import asset

@asset(schedule="@daily")
def daily_sales():
    return extract_sales_data()

@asset(schedule=[daily_sales])
def monthly_sales():
    daily_sales = fetch_daily_sales()
    return aggregate_monthly(daily_sales)
```


from airflow.sdk import asset
@asset(
name="my_asset"
schedule="@daily",
uri="file://path/to/my/asset"
extra={"size_mb": "2"}
group="the_assets"
)
def my_asset():
...

Grouping



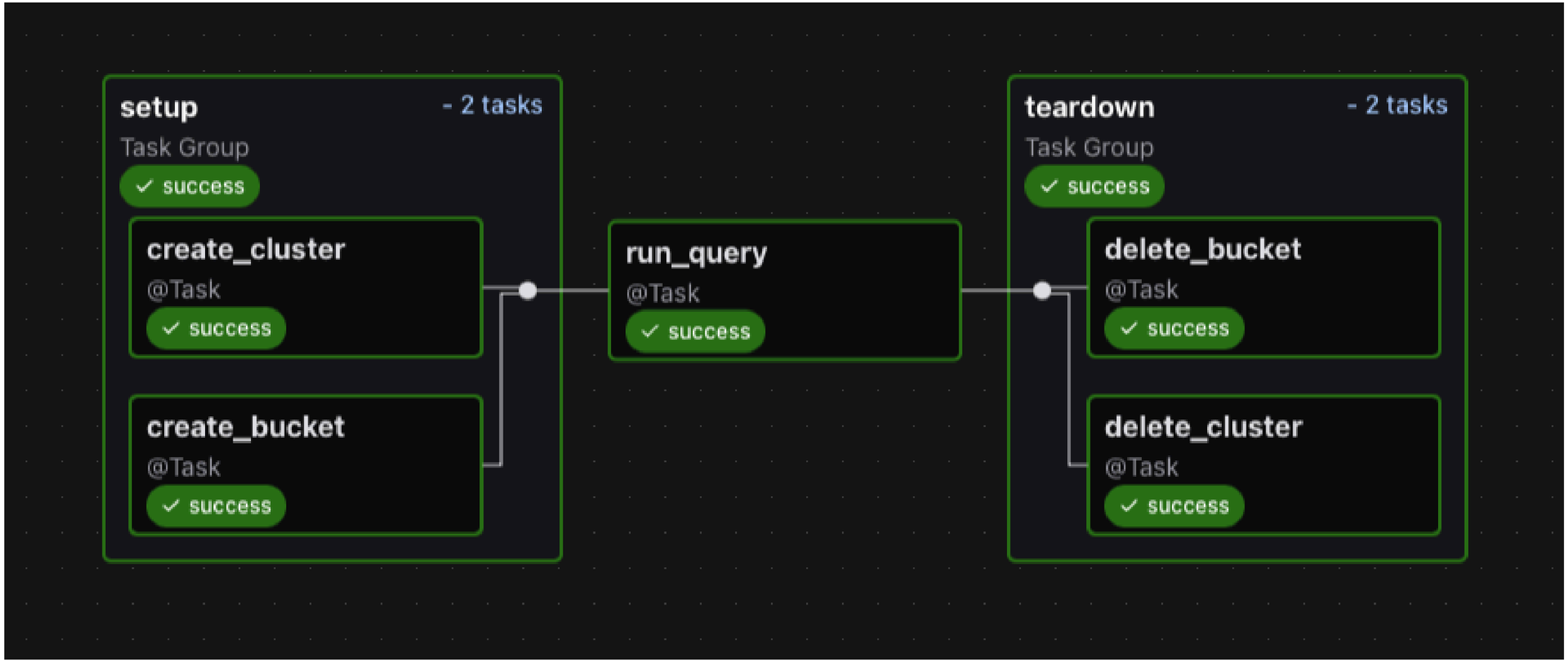
Grouping



Grouping

Using Task Groups allows you to:

- Organize complicated DAGs, visually grouping tasks that belong together in the Airflow UI.
- Apply default_args to sets of tasks, instead of at the DAG level.
- Turn task patterns into modules that can be reused across DAGs or Airflow instances.



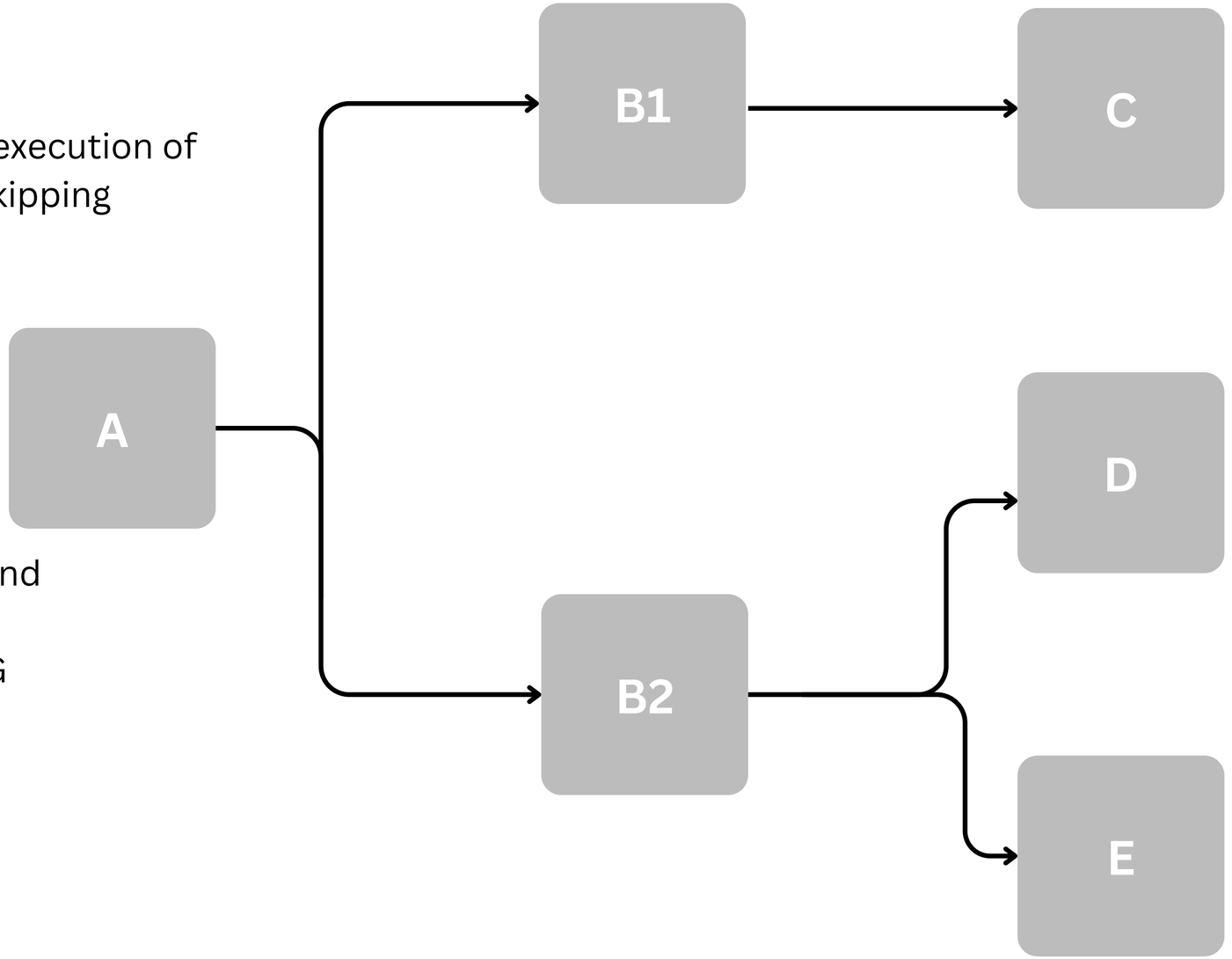
Branching

What is Branching in Airflow?

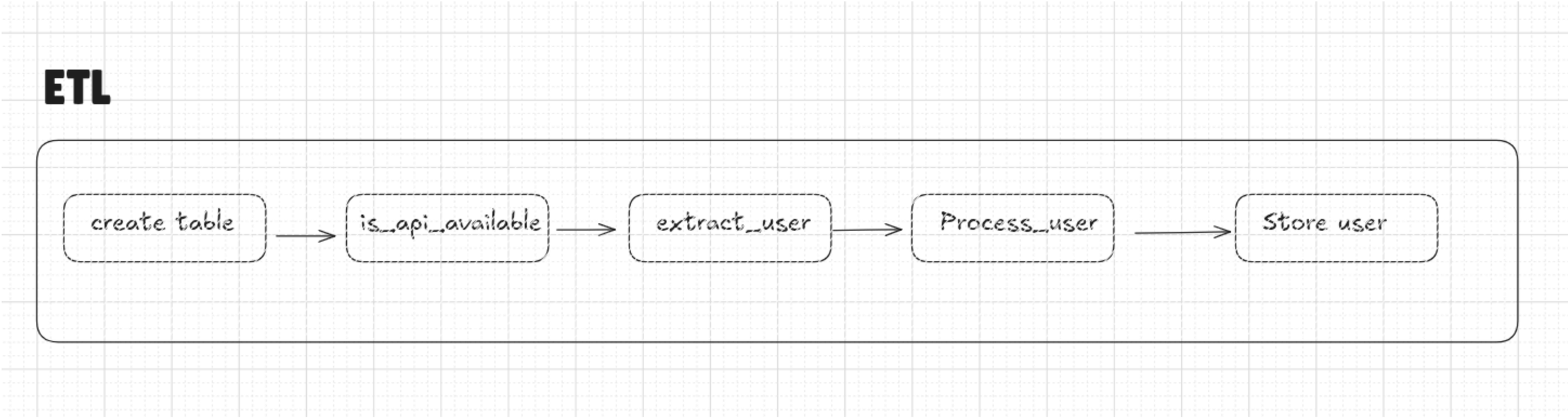
- Branching is the concept of directing the execution of a workflow to one or more tasks — while skipping others — based on some runtime logic.

Why Use Branching?

- Efficiency: Save API calls, DB writes, and compute time.
- Clarity: Make your workflows self-aware and smarter.
- Control: Enable/disable parts of your DAG dynamically.



Small ETL


ETL
create table
is_api_available
extract_user
Process_user
Store user

Demo(ETL)

Thanks